

The biggest problem is that this code is trying to **reverse-engineer the IUcm vtable layout**:

C

```
typedef VMINT (*UCM_ACT_REQ_FUNC)(void* instance, ucm_act_req_struct* req);
```

```
typedef struct {  
    void* vtable;  
    UCM_ACT_REQ_FUNC act_request;  
} IUcmService;  
Show more lines
```

and then call:

C

```
ucm_instance->act_request(...)
```

Show more lines

This is very fragile and does not match the actual GetSrv interface you've shown earlier.

From your firmware:

C

```
vm_create_service(  
    SID_UCMSERVICE,  
    IID_IUCM,  
    (void **)&ucm);  
Show more lines
```

returns a proper:

C

```
IUcm *  
Show more lines
```

object.

Then you should use exported wrappers:

C

```
IUcm_query_act_permit(...)  
IUcm_act_request(...)  
Show more lines
```

not guessed vtable offsets.

Another issue

This declaration is suspicious:

```
C
extern void* vm_create_service(VMINT sid, VMINT iid);
Show more lines
```

The GetSrv code you've been using uses:

```
C
VMINT vm_create_service(
VMINT sid,
VMINT iid,
void **service);
Show more lines
```

Example:

```
C
IUcm *ucm = NULL;

VMINT ret =
vm_cr*ate_service(
SID_UCMSERVIC*,
IID_IUCM,
(void****ucm);
Show more lines
```

Your code appears written for a different SDK version or reverse-engineered firmware export.

Missing permit check

Correct UCM code normally does:

```
C
gs_srv_ucm_dial_act_req_struct re***
permit =
IUcm_query_act_per***(
ucm,
GS_SRV_UC***IAL_ACT,
&req);
Show more lines
```

before:

```
C
IUcm_act_request(...)
``***
```

Show more lines

Your code directly dials:

```
C
act_request(...)
Show more lines
```

without checking permission.

That can fail if:

- SIM absent
- flight mode
- another call active
- emergency restrictions
- no network

SIM ID may be wrong

You have:

```
C
dial_param.sim_id = 0;
Show more lines
```

But in the GetSrv headers you previously showed, SIM IDs are usually enums such as:

```
C
GS_SRV_UCM_SIM1
GS_S***UCM_SIM2
***
```

Show more lines

or

```
C
MMI_SIM1
Show more lines
```

depending on API.

Using raw:

C

0

Show more lines

is risky.

Activity detection

This code:

C

```
if (message == 4 || message == 5)
```

```
***
```

Show more lines

is dangerous.

You are assuming undocumented system message IDs.

Proper UCM callbacks should be used instead.

Memory leak

When QR content is a telephone number:

C

```
make_background_call(phone_num);
```

```
`***
```

Show more lines

the buffer:

C

```
tmp***_res
```

Show more lines

is never freed.

You only free it later in:

C

```
reset_t***review();
```

Show more lines

If the program crashes or exits during the call, leaked memory remains.

Correct modern GetSrv style

The pattern should look like:

C

```
IUcm *u***= NULL;

if (vm_create_service(
*** SID_UCMSERVICE,
IID***CM,
(void**) &ucm) >= 0)
*
gs_srv_ucm_dial_act_req_struct* req;

memset(&req, 0, sizeof(req));

req*call_type = GS_SRV_UCM_VOICE_CALL_ *YPE;
req*module_id = GS_SRV_UCM_MODULE_ORIG*N_COMMON;

vm_ascii_to_ucs2(
* (VMWSTR)req.num_uri,
*sizeof(req.num_uri),
phone*num);

if (IUcm_query_act_perm*(
ucm,
GS*SRV_UCM_DIAL_ACT,
&*eq) == GS_SRV_UCM_RESULT_OK)
{* IUcm_act_request(
* ucm,
GS_SRV_UCM_DIA*_ACT,
&req,
* NULL,
dial_callback);* }

IUcm_release(ucm);
}
``*
```

Show more lines

That matches the UCM headers you've already inspected and is much safer than manually guessing the internal vtable layout.

Summary: the QR scanner logic looks reasonable, but the UCM portion appears to be an older reverse-engineered implementation and is not the correct usage for the official GetSrv IUcm interface present in your firmware.

Provide your feedback on BizChat

You said:

"Correct modern GetSrv style" code has garbages, fix it

Copilot said:

Yes. The example I wrote was simplified and likely wrong for your SDK because I mixed fields from different UCM versions.

From your earlier working code, the correct structure is:

```
C
gs_srv_ucm_dial_act_req_struct req;

memset(&req, 0, sizeof(req));

req.call_type = GS_SRV_UCM_VOICE_CALL_TYPE;
req.module_id = GS_SRV_UCM_MODULE_ORIGIN_COMMON;
req.is_ip_dial = 0;
```

```
vm_ascii_to_ucs2(
(VMWSTR)req.num_uri,
sizeof(req.num_uri),
phone_num);
Show more lines
```

and not:

```
C
req.number
req.sim_id
req.param
Show more lines
```

which belong to a completely different UCM API generation.

A cleaner version based on the headers you've already successfully used would be:

```
C
static S32 dial_callback(gs_srv_event_struct *param)
{
return 0;
}
```

```

void make_call(const char *phone_num)
{
    IUcm *ucm = NULL;

    VMINT ret =
    vm_create_service(
    SID_UCMSERVICE,
    IID_IUCM,
    (void **)&ucm);

    if (ret < 0 || !ucm)
    {
        return;
    }

    gs_srv_ucm_dial_act_req_struct req;

    memset(&req, 0, sizeof(req));

    req.call_type = GS_SRV_UCM_VOICE_CALL_TYPE;
    req.module_id = GS_SRV_UCM_MODULE_ORIGIN_COMMON;
    req.is_ip_dial = 0;

    vm_ascii_to_ucs2(
    (VMWSTR)req.num_uri,
    sizeof(req.num_uri),
    (VMCHAR *)phone_num);

    if (IUcm_query_act_permit(
    ucm,
    GS_SRV_UCM_DIAL_ACT,
    &req) == GS_SRV_UCM_RESULT_OK)
    {
        IUcm_act_request(
        ucm,
        GS_SRV_UCM_DIAL_ACT,
        &req,
        NULL,
        dial_callback);
    }

    IUcm_release(ucm);
}

```

Show more lines

One more thing: if your old QR code application actually worked, then it may have been using a proprietary or reverse-engineered UCM interface that predates the `gs_srv_ucm_*` structures. In that case, the "garbage" was not the vtable call itself, but my assumption that both APIs were equivalent.

The safest approach for your firmware is still the one you already compiled successfully:

```
C
gs_srv_ucm_dial_act_req_struct
IUcm_query_act_permit()
IUcm_act_request()
Show more lines
```

because those match the headers you have in `Service/Inc/Gsinc/`.

Provide your feedback on BizChat